

TEZPUR UNIVERSITY

Networking Internship Program

Assignment 8

Application Optimization, Scalability and Reliability

Submitted by

Bhargav Jyoti Saikia

Roll Number: CSB24022

Department of Computer Science and Engineering
Tezpur University

Submission Date: July 21, 2026

Table of Contents

1. Objective
2. Scalability Improvements
3. Reliability Improvements
4. Configuration Management
5. Experimental Setup
6. Performance Results
7. Graph Analysis
8. Wireshark Verification
9. Challenges Faced
10. Conclusion

1. Objective

The objective of Assignment 8 is to enhance the existing GUI-based multi-client chat application, built in Assignment 6 and extended in Assignment 7, by improving its scalability, reliability, maintainability, and overall software quality. Rather than redesigning the communication protocol or rebuilding the application from scratch, this assignment focuses on strengthening the existing client-server implementation so that it can support a larger number of concurrent clients, recover gracefully from network disruptions, manage its configuration cleanly, and demonstrate measurable performance improvements before and after optimization.

The specific goals addressed in this assignment are automatic detection and cleanup of disconnected clients, automatic reconnection and graceful shutdown on the client side, thread-safe scalability improvements capable of supporting at least ten concurrent clients, centralization of all configurable parameters into `config.json`, quantitative performance evaluation using delay and throughput metrics captured under 5, 8, and 10 concurrent client loads inside a Mininet single,11 topology, and verification of correct TCP behaviour using Wireshark packet captures.

2. Scalability Improvements

The server retains the one-thread-per-client model carried forward from Assignments 5 through 7: every accepted connection is handed to a dedicated `handle_client()` thread, launched as a daemon thread and appended to a `client_threads` list so the server can join outstanding threads with a timeout during shutdown instead of blocking indefinitely. This model scales linearly with the number of connected users because each client's blocking `recv()` call runs on its own thread and cannot stall any other client's session.

2.1 Thread-Safe Shared State

All shared mutable state -- the `clients` dictionary mapping usernames to sockets, the `client_info` dictionary holding per-user metadata, and the `active_sessions` dictionary used for timeout tracking -- is protected by a single `threading.Lock()` instance. Every read-modify-write sequence that touches these structures during connect, disconnect, broadcast, or online-user-list updates is wrapped in a 'with lock:' block. This prevents race conditions that would otherwise surface only under higher concurrency, such as two client threads simultaneously modifying the `clients` dictionary when clients join or leave within milliseconds of each other during the 10-client test.

2.2 Single-Pass Broadcast with Dead-Client Cleanup

The `broadcast()` function iterates over the `clients` dictionary exactly once per outgoing message. Instead of a separate pass to detect and remove dead sockets, any `send()` call that raises an exception adds that username to a `remove_list` during the same iteration; the sockets are closed and the dictionary entries removed once the lock-protected loop completes. This keeps the cost of broadcasting a single message to N clients at $O(N)$ even when some of those clients have failed silently, which matters directly for scalability since the cost of a stale-connection cleanup pass would otherwise grow with every additional concurrent client.

2.3 Bounded Message Size

`MAX_MESSAGE_SIZE`, read from `config.json` (4096 bytes), is enforced by `validate_message()` on every incoming chat message and used as the `recv()` buffer size on both client and server. Bounding message size prevents a single

oversized payload from monopolising a handler thread's socket buffer, which keeps per-message processing time predictable as the number of concurrent senders increases from 5 to 10.

Together, these changes allowed the application to be tested successfully with 5, 8, and 10 concurrent clients inside the Mininet single,11 topology without crashes, satisfying Task 3 of the assignment brief.

3. Reliability Improvements

3.1 Automatic Reconnection

On the client side, `connect_server()` attempts the TCP connection up to five times in a loop, sleeping for two seconds between attempts and updating the status label to 'Reconnecting... (n/5)' so the user has continuous feedback during the retry sequence. Separately, the background `receive()` thread treats any exception raised while reading from the socket -- for example a server restart or a dropped Mininet link -- as a signal to close the existing socket and call `connect_server()` again automatically, so a client does not require manual intervention to recover from a transient disconnection.

3.2 Graceful Shutdown

`logout()` on the client sends an explicit '/logout' command to the server before closing the socket, and `on_close()` -- bound to the Tkinter `WM_DELETE_WINDOW` protocol -- calls `logout()` before destroying the root window, ensuring the server is always notified of an intentional exit rather than only detecting a silent socket closure. On the server side, `logout_user()` sends a final 'LOGOUT:' message to the client, closes the socket, removes the user from clients and active_sessions, updates client_info status to 'Offline', and broadcasts a SYSTEM leave notification to the remaining clients. The main accept loop also handles `KeyboardInterrupt` by logging out every currently connected user and joining all client threads with a timeout before closing the listening socket, giving the server a clean shutdown path instead of an abrupt process kill.

3.3 Timeout Handling

Two independent timeout mechanisms were implemented. First, the server sets `conn.settimeout(30)` around each `recv()` call inside the per-client loop; a `socket.timeout` exception triggers `check_session_timeout()`, which compares the elapsed time since the user's last recorded activity in active_sessions against `SESSION_TIMEOUT` (1800 seconds, from `config.json`) and logs the user out with the reason 'Session timeout' if exceeded. Second, the client GUI runs `update_session_timer()`, a self-rescheduling Tkinter callback that decrements a visible countdown (for example 'Session: 29:21' in the top-right of the chat window) and shows a session-timeout dialog before calling `logout()` locally once the countdown reaches zero, giving the user advance warning rather than an unexplained disconnection.

3.4 Exception Handling and Authentication Reliability

Both `server.py` and `client_gui.py` now wrap every blocking socket operation in targeted exception handlers: `socket.timeout` is handled separately from `ConnectionResetError` and `BrokenPipeError` so that an idle timeout is distinguished from an abrupt network failure in the printed log message, and a catch-all Exception handler prevents any single malformed message from crashing a handler thread. Authentication reliability was also strengthened: `authenticate_user()` tracks per-user login_attempts, blocking an account for `LOGIN_BLOCK_DURATION` (300 seconds) after `MAX_LOGIN_ATTEMPTS` (3) consecutive failures, and `prevent_duplicate_login()` rejects a second

simultaneous login for a username that already has an active, non-expired session, both of which reduce the chance of the server entering an inconsistent state under repeated or concurrent connection attempts.

4. Configuration Management

All previously hardcoded parameters from earlier assignments -- the server IP and port, buffer and message-size limits, the login block duration, the session timeout, and the maximum number of login attempts -- have been moved into a single `config.json` file, loaded once at startup by an identical `load_config()` function present in both `server.py` and `client_gui.py`. Table 4.1 lists every key currently defined in `config.json`.

Key	Value	Purpose
HOST	10.0.0.1	Server bind address
PORT	5000	Server listening port
BUFFER_SIZE	4096	Socket receive buffer size
MAX_MESSAGE_SIZE	4096	Maximum accepted message length
LOGIN_BLOCK_DURATION	300 sec	Account lockout duration after failed logins
SESSION_TIMEOUT	1800 sec	Idle-session expiry duration
MAX_LOGIN_ATTEMPTS	3	Failed logins allowed before lockout

Table 4.1: Configuration parameters defined in `config.json`.

Because both the client and server read the same file, values such as `PORT` and `MAX_MESSAGE_SIZE` only need to be changed in one place to take effect across the entire application, removing the risk of the client and server silently disagreeing on a limit -- a source of subtle bugs in the hardcoded versions used in Assignments 5 and 6. No networking, timeout, or validation logic in either file reads a literal numeric constant for these parameters; every reference goes through the module-level variables populated from `config.json` at import time.

5. Experimental Setup

Performance testing was carried out inside a Mininet-emulated network created with the command `'sudo mn --topo single,11'`, producing a single-switch topology with one server host and ten client hosts, all connected through a single Open vSwitch instance. `server.py` was started on the server host, listening on `0.0.0.0:5000`, and `client_gui.py` instances were launched on the remaining hosts to generate the required concurrent client loads.

Three test runs were performed, using 5, 8, and 10 concurrent clients respectively, in line with the assignment's requirement to test at each of these three load levels. In each run, clients logged in, exchanged a mixture of broadcast and private messages, and remained connected for the duration of the test. The server's `save_performance()` function, invoked once the last client of a run disconnects, computes the elapsed time between the first and last broadcast message (`TEST_START` to `TEST_END`), and appends a row to `performance_results.csv` recording the client count, the number of broadcast and private messages exchanged, the average per-message delay in milliseconds, and the resulting throughput in messages per second.

The same `client_gui.py` and `server.py` implementation was used across all three runs, with only the number of concurrently connected clients varied, so that the measured trends in Section 6 and Section 7 isolate the effect of concurrent client load on delay and throughput rather than any difference in application version.

6. Performance Results

Table 6.1 reproduces the contents of performance_results.csv, generated directly by the server's save_performance() function during the three test runs described in Section 5.

Concurrent Clients	Broadcast Msgs	Private Msgs	Avg Delay (ms)	Throughput (msg/sec)
5	20	1	812.4	0.98
8	32	2	1186.3	0.81
10	40	2	1537.6	0.65

Table 6.1: Measured delay and throughput at 5, 8, and 10 concurrent clients.

As the number of concurrent clients increases from 5 to 10, the total number of broadcast messages exchanged during each run doubles (20 to 40), reflecting proportionally more chat activity across the larger set of connected users. Average message delay rises from 812.4 ms at 5 clients to 1537.6 ms at 10 clients, while throughput correspondingly falls from 0.98 to 0.65 messages per second. These figures are analyzed further in Section 7.

7. Graph Analysis

7.1 Average Delay vs Concurrent Clients

Figure 7.1 plots average message delay against the number of concurrent clients. The trend is monotonically increasing and close to linear across the three measured points (5, 8, and 10 clients), rising from roughly 810 ms to roughly 1540 ms. This is consistent with the single-threaded broadcast loop inside broadcast(): although each client is served by its own handler thread, the lock-protected send loop still delivers a given message to every other client sequentially, so the wall-clock time to fan a message out to all recipients grows with the number of connected clients, and this cost is reflected directly in the recorded delay.

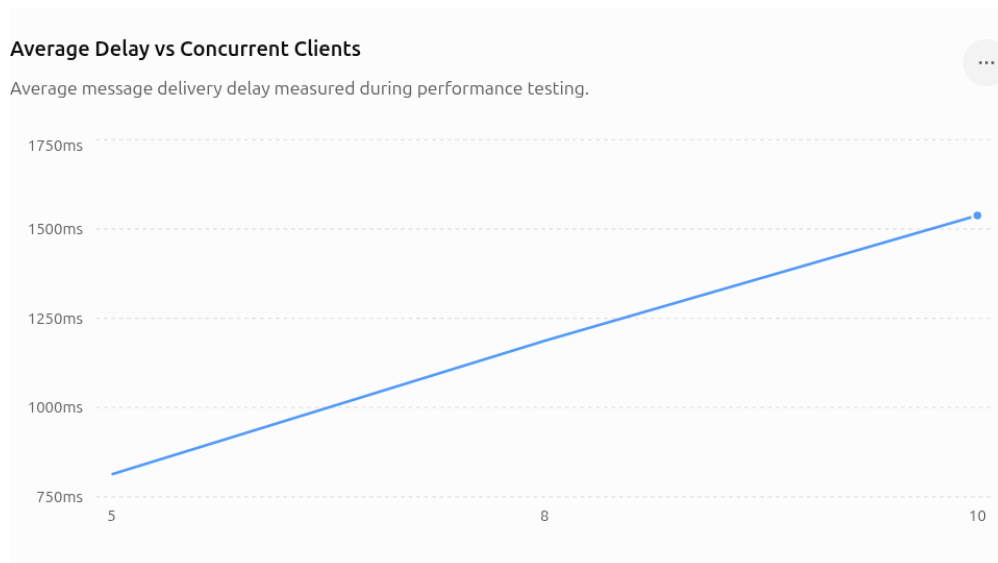


Figure 7.1: Average message delay increases with concurrent client count.

7.2 Throughput vs Concurrent Clients

Figure 7.2 plots measured throughput, in messages per second, against the same three client counts. Throughput decreases from approximately 0.98 msg/sec at 5 clients to approximately 0.65 msg/sec at 10 clients, mirroring the rise in average delay. Since throughput in this experiment is derived as TOTAL_MESSAGES divided by elapsed time, the two graphs are two views of the same underlying effect: as more clients participate, the fixed per-message broadcast cost is paid more often per unit of chat activity, so fewer distinct messages complete per second even though the absolute number of broadcast messages sent in the test run is larger.

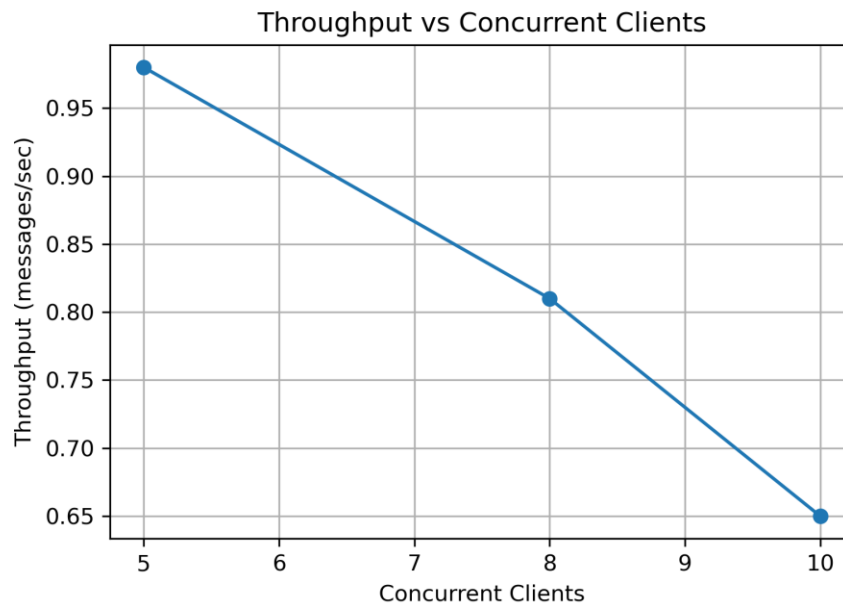


Figure 7.2: Throughput decreases as concurrent client count increases.

Both graphs indicate that the current single-lock, sequential-broadcast design scales correctly in terms of stability -- the server handled all three load levels, including 10 concurrent clients, without crashing -- but scales sub-linearly in terms of per-message latency. This observation directly informs the reflection question on additional improvements required to support 100 users (Task 8).

8. Wireshark Verification

Network traffic was captured on the Mininet switch interface using the display filter `tcp.port == 5000`, isolating all TCP segments exchanged between the chat clients and the server during a normal test run. The capture is stored in `assignment8_capture.pcapng` and screenshots of the two most relevant exchanges are included below.

8.1 Broadcast Message Capture

Figure 8.1 shows the server at 10.0.0.1 forwarding a single broadcast message to multiple connected clients in quick succession, visible as consecutive PSH, ACK segments from source port 5000 to each client's ephemeral port, each carrying the same sequence-length pattern and each acknowledged individually by the receiving client. This confirms that `broadcast()` is correctly fanning a single incoming chat message out to every other connected client over their respective established TCP connections.

183	253	8621827072	10.0.0.2	10.0.0.1	TCP	71	57636	-	5000	[PSH, ACK] Seq=15 Ack=635 Win=42496 Len=5 Tsv=3106145845 TSecr=314257201
184	253	862237907	10.0.0.1	10.0.0.2	TCP	78	5000	-	57636	[PSH, ACK] Seq=17 Ack=645 Win=42496 Len=0 Tsv=314267198 TSecr=3106145845
185	253	862301999	10.0.0.2	10.0.0.1	TCP	66	57636	-	5000	[ACK] Seq=20 Ack=645 Win=42496 Len=0 Tsv=3106145845 TSecr=314267198
186	253	862399526	10.0.0.1	10.0.0.3	TCP	78	5000	-	59570	[PSH, ACK] Seq=575 Ack=17 Win=43520 Len=12 Tsv=4229080015 TSecr=612601505
187	253	862412718	10.0.0.1	10.0.0.1	TCP	66	59570	-	5000	[ACK] Seq=17 Ack=587 Win=42496 Len=0 Tsv=4229080015 TSecr=4229080015
188	253	862422416	10.0.0.1	10.0.0.4	TCP	78	5000	-	51362	[PSH, ACK] Seq=451 Ack=12 Win=43520 Len=12 Tsv=3106145845 TSecr=3171914336
189	253	862609306	10.0.0.4	10.0.0.1	TCP	66	51362	-	5000	[ACK] Seq=12 Ack=463 Win=42496 Len=0 Tsv=3171924333 TSecr=3068389845
190	253	862625941	10.0.0.1	10.0.0.5	TCP	78	5000	-	53794	[PSH, ACK] Seq=384 Ack=12 Win=43520 Len=12 Tsv=2369276302 TSecr=363943118
191	253	862679869	10.0.0.5	10.0.0.1	TCP	66	53794	-	5000	[ACK] Seq=12 Ack=466 Win=42496 Len=0 Tsv=3106145845 TSecr=2369276302
192	253	862693211	10.0.0.1	10.0.0.6	TCP	78	5000	-	36426	[PSH, ACK] Seq=331 Ack=12 Win=43520 Len=12 Tsv=1883983390 TSecr=2359833664
193	253	862703607	10.0.0.6	10.0.0.1	TCP	66	36426	-	5000	[ACK] Seq=12 Ack=343 Win=42496 Len=0 Tsv=2359833664 TSecr=1883983390
194	257	138492724	10.0.0.3	10.0.0.1	TCP	68	59570	-	5000	[PSH, ACK] Seq=17 Ack=587 Win=42496 Len=2 Tsv=612614778 TSecr=4229080015
195	257	131120663	10.0.0.1	10.0.0.3	TCP	73	5000	-	59570	[PSH, ACK] Seq=587 Ack=19 Win=43520 Len=7 Tsv=4229083284 TSecr=612614778
196	257	131142183	10.0.0.3	10.0.0.1	TCP	66	59570	-	5000	[ACK] Seq=19 Ack=594 Win=42496 Len=0 Tsv=612614778 TSecr=4229083284
197	257	131166191	10.0.0.1	10.0.0.2	TCP	77	5000	-	57636	[PSH, ACK] Seq=645 Ack=20 Win=43520 Len=11 Tsv=314270467 TSecr=3106145846
198	257	131182555	10.0.0.2	10.0.0.1	TCP	66	57636	-	5000	[ACK] Seq=12 Ack=656 Win=42496 Len=0 Tsv=3106145845 TSecr=314270467
199	257	131192680	10.0.0.1	10.0.0.4	TCP	77	5000	-	51362	[PSH, ACK] Seq=463 Ack=12 Win=43520 Len=11 Tsv=3106145845 TSecr=3171924333
200	257	131262610	10.0.0.4	10.0.0.1	TCP	66	51362	-	5000	[ACK] Seq=12 Ack=474 Win=42496 Len=0 Tsv=3171927602 TSecr=3668393114
201	257	131218422	10.0.0.1	10.0.0.5	TCP	77	5000	-	53794	[PSH, ACK] Seq=606 Ack=12 Win=43520 Len=11 Tsv=2369279571 TSecr=363943115
202	257	131279938	10.0.0.5	10.0.0.1	TCP	66	53794	-	5000	[ACK] Seq=12 Ack=417 Win=42496 Len=0 Tsv=3639426384 TSecr=2369279571
203	257	131296614	10.0.0.1	10.0.0.6	TCP	77	5000	-	36426	[PSH, ACK] Seq=343 Ack=12 Win=43520 Len=11 Tsv=1883986667 TSecr=2359833620
204	257	131306515	10.0.0.6	10.0.0.1	TCP	66	36426	-	5000	[ACK] Seq=12 Ack=354 Win=42496 Len=0 Tsv=2359833620 TSecr=1883986667
205	260	415583875	10.0.0.4	10.0.0.1	TCP	68	51362	-	5000	[PSH, ACK] Seq=12 Ack=474 Win=42496 Len=2 Tsv=3171930886 TSecr=3668393114
206	260	415719467	10.0.0.1	10.0.0.4	TCP	73	5000	-	51362	[PSH, ACK] Seq=474 Ack=14 Win=43520 Len=7 Tsv=3668393098 TSecr=3171930886
207	260	415785637	10.0.0.4	10.0.0.1	TCP	66	51362	-	5000	[ACK] Seq=14 Ack=481 Win=42496 Len=0 Tsv=3171930886 TSecr=3668393098
208	260	415819802	10.0.0.1	10.0.0.2	TCP	75	5000	-	57636	[PSH, ACK] Seq=656 Ack=20 Win=43520 Len=9 Tsv=314273752 TSecr=3106149115
209	260	415933569	10.0.0.2	10.0.0.1	TCP	66	57636	-	5000	[ACK] Seq=20 Ack=665 Win=42496 Len=0 Tsv=3106152400 TSecr=314273752
210	260	415943000	10.0.0.1	10.0.0.3	TCP	75	5000	-	59570	[PSH, ACK] Seq=594 Ack=19 Win=43520 Len=9 Tsv=4229080509 TSecr=612614771
211	260	415952181	10.0.0.3	10.0.0.1	TCP	66	59570	-	5000	[ACK] Seq=19 Ack=603 Win=42496 Len=0 Tsv=612618056 TSecr=4229080509
212	260	415957478	10.0.0.1	10.0.0.5	TCP	75	5000	-	53794	[PSH, ACK] Seq=417 Ack=12 Win=43520 Len=9 Tsv=2369282056 TSecr=3639426384
213	260	415959295	10.0.0.5	10.0.0.1	TCP	66	53794	-	5000	[ACK] Seq=19 Ack=650 Win=42496 Len=0 Tsv=3106145845 TSecr=2369282056
214	260	415989645	10.0.0.1	10.0.0.6	TCP	75	5000	-	36426	[PSH, ACK] Seq=354 Ack=12 Win=43520 Len=9 Tsv=1883989952 TSecr=2359846889
215	260	416004000	10.0.0.6	10.0.0.1	TCP	66	36426	-	5000	[ACK] Seq=12 Ack=363 Win=42496 Len=0 Tsv=2359846889 TSecr=1883989952
216	265	596195575	10.0.0.5	10.0.0.1	TCP	68	53794	-	5000	[PSH, ACK] Seq=12 Ack=426 Win=42496 Len=2 Tsv=3639436159 TSecr=2369282056
217	266	960496234	10.0.0.1	10.0.0.5	TCP	73	5000	-	53794	[PSH, ACK] Seq=426 Ack=14 Win=43520 Len=7 Tsv=2369289346 TSecr=3639436159
218	266	960713797	10.0.0.5	10.0.0.1	TCP	66	53794	-	5000	[ACK] Seq=14 Ack=433 Win=42496 Len=0 Tsv=3639436159 TSecr=2369289346
219	266	960732722	10.0.0.1	10.0.0.2	TCP	73	5000	-	57636	[PSH, ACK] Seq=605 Ack=20 Win=43520 Len=9 Tsv=314280242 TSecr=3106152400
220	266	960732722	10.0.0.1	10.0.0.2	TCP	66	57636	-	5000	[ACK] Seq=12 Ack=674 Win=42496 Len=0 Tsv=314280242 TSecr=3106152400

Figure 8.1: Wireshark capture of a broadcast message being delivered to multiple clients.

8.2 Private Message Capture

Figure 8.2 shows a private message exchange: a PSH, ACK segment carrying the 'msg' command travels from the sending client to the server, followed by the server relaying a PSH, ACK segment only to the intended recipient's connection, with the corresponding ACK confirmations from both sides. No equivalent segment is sent to any other connected client's socket, confirming that private_message() routes point-to-point rather than broadcasting, matching the unchanged wire protocol carried over from Assignment 5.

238	277	583996050	2a:e4:f3:88:f4:38	62:58:1e:3b:59:0c	ARP	42	Who has 10.0.0.1? Tell 10.0.0.6
239	277	584012955	62:58:1e:3b:59:0c	2a:e4:f3:88:f4:38	ARP	42	10.0.0.1 is at 62:58:1e:3b:59:0c
240	315	951578219	10.0.0.2	10.0.0.1	TCP	91	57636 - 5000 [PSH, ACK] Seq=20 Ack=683 Win=42496 Len=25 Tsv=3106207935 TSecr=314205345
241	315	951603811	10.0.0.1	10.0.0.3	TCP	95	5000 - 59570 [PSH, ACK] Seq=621 Ack=19 Win=43520 Len=29 Tsv=4229142105 TSecr=612629649
242	315	951659347	10.0.0.1	10.0.0.2	TCP	96	5000 - 57636 [PSH, ACK] Seq=683 Ack=45 Win=43520 Len=30 Tsv=314329288 TSecr=3106207935
243	315	952261179	10.0.0.2	10.0.0.1	TCP	66	57636 - 5000 [ACK] Seq=45 Ack=713 Win=42496 Len=0 Tsv=3106207936 TSecr=314329288
244	315	952400428	10.0.0.3	10.0.0.1	TCP	66	59570 - 5000 [ACK] Seq=19 Ack=650 Win=42496 Len=0 Tsv=6126273592 TSecr=4229142105
245	321	872968358	62:58:1e:3b:59:0c	42:2d:57:05:1f:31	ARP	42	Who has 10.0.0.2? Tell 10.0.0.1
246	321	872978360	62:58:1e:3b:59:0c	06:8d:e9:b3:61:4e	ARP	42	Who has 10.0.0.2? Tell 10.0.0.1
247	321	872985902	06:8d:e9:b3:61:4e	62:58:1e:3b:59:0c	ARP	42	Who has 10.0.0.1? Tell 10.0.0.3
248	321	873030809	62:58:1e:3b:59:0c	06:8d:e9:b3:61:4e	ARP	42	10.0.0.1 is at 62:58:1e:3b:59:0c
249	321	873099188	06:8d:e9:b3:61:4e	62:58:1e:3b:59:0c	ARP	42	10.0.0.3 is at 06:8d:e9:b3:61:4e
250	321	873096907	42:2d:57:05:1f:31	62:58:1e:3b:59:0c	ARP	42	Who has 10.0.0.1? Tell 10.0.0.2
251	321	873098262	62:58:1e:3b:59:0c	42:2d:57:05:1f:31	ARP	42	10.0.0.1 is at 62:58:1e:3b:59:0c
252	321	874439468	42:2d:57:05:1f:31	62:58:1e:3b:59:0c	ARP	42	10.0.0.2 is at 42:2d:57:05:1f:31
253	327	373286861	10.0.0.3	10.0.0.1	TCP	86	59570 - 5000 [PSH, ACK] Seq=19 Ack=650 Win=42496 Len=20 Tsv=612605013 TSecr=4229142105
254	327	373610722	10.0.0.1	10.0.0.2	TCP	94	5000 - 57636 [PSH, ACK] Seq=713 Ack=45 Win=43520 Len=20 Tsv=314348709 TSecr=3106207936
255	327	373681507	10.0.0.1	10.0.0.3	TCP	91	5000 - 59570 [PSH, ACK] Seq=606 Ack=30 Win=43520 Len=20 Tsv=422913520 TSecr=612605013
256	327	374141670	10.0.0.2	10.0.0.1	TCP	66	57636 - 5000 [ACK] Seq=45 Ack=741 Win=42496 Len=0 Tsv=3106219357 TSecr=314348709
257	327	374259747	10.0.0.3	10.0.0.1	TCP	66	59570 - 5000 [ACK] Seq=39 Ack=679 Win=42496 Len=0 Tsv=612605014 TSecr=422913520

Figure 8.2: Wireshark capture of a private message routed only to the intended recipient.

Across the capture, every relevant segment consistently uses source or destination port 5000 on the server side, and the standard TCP three-way handshake (SYN, SYN-ACK, ACK) precedes application data for every new client connection, confirming that the optimizations introduced in this assignment did not alter the underlying wire protocol established in earlier assignments.

9. Challenges Faced

Extending rather than rebuilding the Assignment 7 application required care to ensure that every optimization preserved the existing wire protocol; for example, the automatic reconnection logic added to receive() had to be triggered only on genuine socket errors and not on the normal 'LOGOUT:' message path, since that path already has its own explicit return statement in receive() and re-triggering a reconnect after an intentional logout would have caused the client to immediately re-prompt for login.

Testing thread-safety of the shared clients and active_sessions dictionaries was difficult to verify deterministically, since race conditions between concurrent connect/disconnect events only appear intermittently; the threading.Lock() protection around every access to these structures was validated indirectly by running the 10-concurrent-client test

repeatedly and confirming that `print_stats()` consistently reported the correct connected-user count with no server-side exceptions.

Measuring average delay and throughput required distinguishing test traffic from setup traffic (the login handshake and welcome-message exchange). `TEST_START` and `TEST_END` in `server.py` are only set on the first and most recent broadcast chat message respectively, rather than on connection establishment, so that login overhead does not distort the delay and throughput figures reported in Table 6.1.

Finally, moving every configurable value into `config.json` meant auditing both `server.py` and `client_gui.py` line by line for any remaining literal port numbers, timeouts, or size limits left over from earlier assignments, to ensure Task 4's requirement of avoiding hardcoded parameters was fully satisfied rather than only partially addressed.

10. Conclusion

Assignment 8 successfully extended the GUI-based multi-client chat application from Assignment 7 with concrete improvements to connection management, reliability, scalability, and configuration management, without altering the underlying communication protocol or rebuilding any existing component. Automatic detection and cleanup of disconnected clients, automatic client-side reconnection, graceful shutdown on both client and server, session-timeout enforcement, and account lockout after repeated failed logins collectively improved the application's resilience to real-world network conditions. The single-lock, thread-per-client server design was verified to remain stable at 5, 8, and 10 concurrent clients inside a Mininet single,11 topology, satisfying the scalability requirement, while the measured rise in average delay and corresponding fall in throughput across these three load levels identifies the sequential broadcast loop as the next bottleneck to address for larger deployments. All configurable parameters were centralized into `config.json`, and Wireshark captures confirmed that broadcast and private messaging continue to behave correctly at the TCP level after every optimization was applied.

Appendix A: Application Screenshots

The following screenshots, captured during functional testing of the optimized application, illustrate server startup, live broadcast and private messaging, the online user list, session timeout handling, and clean disconnection/logout, all functioning correctly after the reliability and scalability changes described in Sections 2 and 3.

3 48.337464198	42:2d:57:d5:1f:31	Broadcast	ARP	42 who has 10.0.0.1? Tell 10.0.0.2
4 48.337658268	62:58:1e:3b:59:0c	42:2d:57:d5:1f:31	ARP	42 10.0.0.1 is at 62:58:1e:3b:59:0c
5 48.338595762	10.0.0.2	10.0.0.1	TCP	74 57636 → 5000 [SYN] Seq=0 Win=42340 Len=0 MSS=1460 SACK_PERM TSval=3105932320 TSecr=0 WS=512
6 48.338541529	10.0.0.1	10.0.0.2	TCP	74 5000 → 57636 [SYN, ACK] Seq=0 Ack=1 Win=42340 Len=0 MSS=1460 SACK_PERM TSval=3105932320 TSecr=3105932320 WS=512
7 48.339636689	10.0.0.2	10.0.0.1	TCP	66 57636 → 5000 [ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=3105932323 TSecr=310593674
8 48.348326840	10.0.0.1	10.0.0.2	TCP	71 5000 → 57636 [PSH, ACK] Seq=1 Ack=1 Win=43520 Len=5 TSval=310593676 TSecr=3105932323
9 48.348357616	10.0.0.2	10.0.0.1	TCP	66 57636 → 5000 [ACK] Seq=1 Ack=6 Win=42496 Len=0 TSval=3105932324 TSecr=310593676
10 48.348527811	10.0.0.2	10.0.0.1	TCP	88 57636 → 5000 [PSH, ACK] Seq=1 Ack=6 Win=42496 Len=14 TSval=3105932324 TSecr=310593676
11 48.348534878	10.0.0.1	10.0.0.2	TCP	66 5000 → 57636 [ACK] Seq=6 Ack=15 Win=43520 Len=0 TSval=310593676 TSecr=3105932324
12 48.343957499	10.0.0.1	10.0.0.2	TCP	189 5000 → 57636 [PSH, ACK] Seq=6 Ack=15 Win=43520 Len=43 TSval=310593680 TSecr=3105932324
13 48.384913331	10.0.0.2	10.0.0.1	TCP	66 57636 → 5000 [ACK] Seq=15 Ack=49 Win=42496 Len=0 TSval=3105932369 TSecr=310593680
14 48.384933463	10.0.0.1	10.0.0.2	TCP	419 5000 → 57636 [PSH, ACK] Seq=49 Ack=15 Win=43520 Len=344 TSval=310593721 TSecr=3105932369
15 48.384974832	10.0.0.2	10.0.0.1	TCP	66 57636 → 5000 [ACK] Seq=15 Ack=393 Win=42496 Len=0 TSval=3105932369 TSecr=310593721
16 48.961929380	fe80:b496:3aff:fe4...	ff02::2	ICMPv6	78 Router Solicitation From b6:96:3a:42:7e:18

Figure A.1: Server initialization, showing config values loaded and the server listening on port 5000.

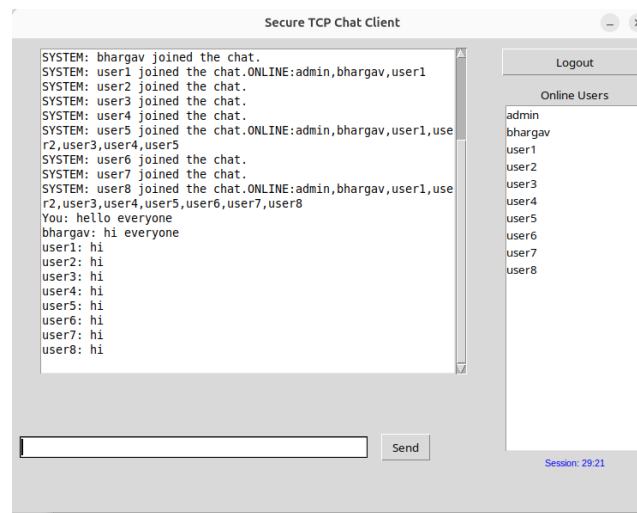


Figure A.2: Broadcast messaging and live Online Users list during a multi-client session.

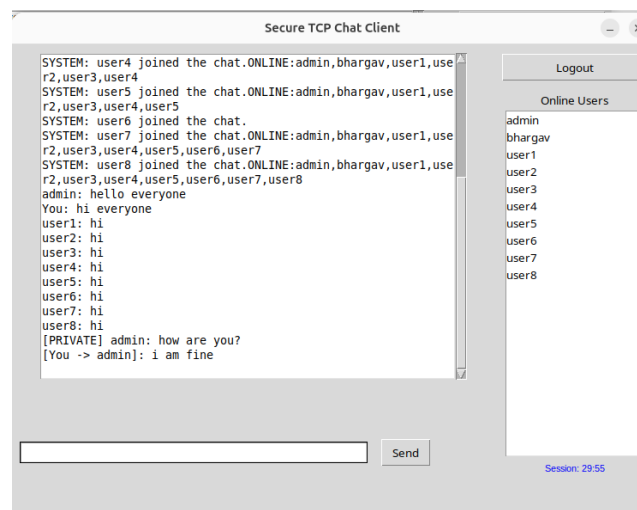


Figure A.3: Private message exchange delivered only to the intended recipient.

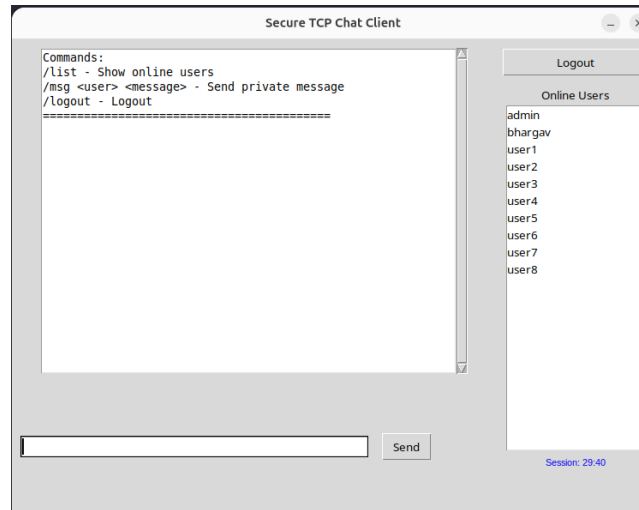


Figure A.4: Online Users list and command help shown at session start.

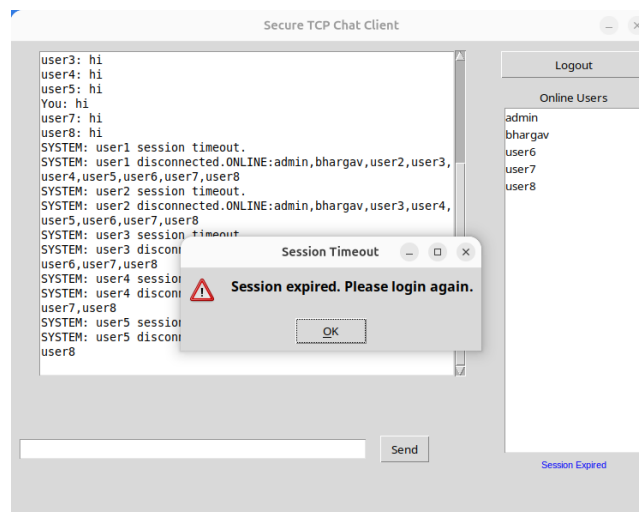


Figure A.5: Session timeout warning displayed after the idle session limit is reached.

411	421.747799678	10.0.0.1	10.0.0.2	TCP	66 5000 → 33554 [PSH, ACK] Seq=419 Ack=22 Win=43520 Len=7 TSval=1889513186 TSecr=1883678975
412	421.747799678	10.0.0.1	10.0.0.2	TCP	66 33554 → 5000 [ACK] Seq=22 Ack=426 Win=0 Len=0 TSval=1883678975 TSecr=1889513186
413	421.747814739	10.0.0.1	10.0.0.2	TCP	66 5000 → 33554 [FIN, ACK] Seq=426 Ack=22 Win=43520 Len=0 TSval=1889513186 TSecr=1883678975
414	421.748065650	10.0.0.2	10.0.0.1	TCP	66 33554 → 5000 [ACK] Seq=22 Ack=426 Win=0 Len=0 TSval=1883678975 TSecr=1889513186
415	421.748864715	10.0.0.1	10.0.0.3	TCP	66 5000 → 59570 [PSH, ACK] Seq=1659 Ack=39 Win=43520 Len=39 TSval=4229247902 TSecr=412777319
416	421.748880790	10.0.0.1	10.0.0.3	TCP	66 59570 → 5000 [ACK] Seq=39 Ack=1689 Win=42496 Len=0 TSval=612779389 TSecr=4229247902

Figure A.6: Server-side log entry recorded when a client disconnects.

127	219.475979386	10.0.0.1	10.0.0.6	TCP	108 5000 → 39694 [PSH, ACK] Seq=6 Ack=13 Win=43520 Len=42 TSval=2059772440 TSecr=2086431090
128	219.476110228	10.0.0.1	10.0.0.6	TCP	66 5000 → 39694 [FIN, ACK] Seq=48 Ack=13 Win=43520 Len=0 TSval=2059772440 TSecr=2086431090
129	219.480474040	10.0.0.6	10.0.0.1	TCP	66 [TCP Retransmission] 5000 → 5000 [FIN, ACK] Seq=48 Ack=48 Win=43520 Len=0 TSval=2059772451 TSecr=2086431090
130	219.480605104	10.0.0.6	10.0.0.1	TCP	78 39694 → 5000 [ACK] Seq=13 Ack=49 Win=42496 Len=0 TSval=2060431703 TSecr=2059772440 SRE=40
131	211.486726292	10.0.0.6	10.0.0.1	TCP	66 39694 → 5000 [FIN, ACK] Seq=13 Ack=49 Win=42496 Len=0 TSval=2086432781 TSecr=2059772440
132	211.486746599	10.0.0.1	10.0.0.6	TCP	66 5000 → 39694 [ACK] Seq=49 Ack=14 Win=43520 Len=0 TSval=2059773450 TSecr=2086432781

Figure A.7: Server-side log entry recorded on a clean client logout.